In-Context Exploration for Efficient Reinforcement Learning in Robotic Manipulation

Nikita Maurer[*,1], **Rickmer Krohn**[*,1,2,3], **Vignesh Prasad**[1,3,4], **Gabriele Tiboni**[1] **and Georgia Chalvatzaki**[1,3,4]

[1] Interactive Robot Perception & Learning, TU Darmstadt
[2] Hessian AI, [3] Robotics Institute Germany
https://ice-exploration.github.io

Abstract: Efficient Reinforcement Learning (RL) depends on the agent's ability to explore its environment. Intrinsic motivation methods explore without a task reward using, e.g., prediction errors. As the behavior of agents trained with intrinsic exploration objectives changes slowly throughout training, it remains unclear how to generate data to initialize RL for efficient robotic manipulation. We introduce In-Context Exploration (ICE) to distill reward-free exploration behaviors from intrinsic exploration agents into a single Transformer model. ICE generates diverse and task-relevant data by changing context from replay or self-generated interactions. Our extensive evaluation across three intrinsic exploration objectives and three manipulation tasks shows that exploration data leads to efficient downstream task learning. ICE data improves task learning across multiple intrinsic objectives and enables near-optimal performance with half the environment steps compared to learning from random data. Furthermore, we transfer policies from the cube-pushing task to the real world, demonstrating improved performance for models trained with ICE exploration data.

1 Introduction

Robot manipulation requires informative physical interactions, yet collecting them is costly and a central obstacle to sample-efficient real-world reinforcement learning [1]. Before a downstream task is specified, its reward cannot guide collection. Intrinsic objectives such as Random Network Distillation (RND) [2], the Intrinsic Curiosity Module (ICM) [3], and dynamics-model disagreement [4] instead reward novelty or prediction uncertainty. Their policies and reward models evolve throughout training, producing a history of potentially useful object interactions.

Existing work transfers this experience through pretrained network parameters, as in URLB [5], or through reward-relabeled exploration datasets, as in ExORL and Explore2Offline [6, 7]. Behavioral Exploration learns a history-conditioned collector from demonstrations using explicit exploration conditioning [8]. These approaches motivate a complementary question: can autonomous intrinsic exploration methods be used to learn a task-agnostic exploration prior that retains behavior to generate diverse and task-relevant data for robotic manipulation tasks without demonstration data or explicit diversity labels?

We introduce **I**n-**C**ontext **E**xploration (ICE), a transformer model trained on state-action pairs from the replay history of an intrinsically motivated exploration agent. ICE distills the diverse exploratory behaviors produced throughout training into a single, context-conditioned transformer. The action is predicted while conditioned on the past episodes, the current rollout transitions, and the current state. ICE is trained on past context and changes its conditioning between rollouts to learn across the evolving behavioral repertoire and generate diverse data.

At deployment, ICE remains frozen and generates trajectories through autoregressive interaction with the environment, updating its context as new state–action pairs are collected. Context can elicit different behaviors from the learned behaviors without changing or switching between models. The generated data is labeled with downstream task rewards and used to initialize the replay buffer

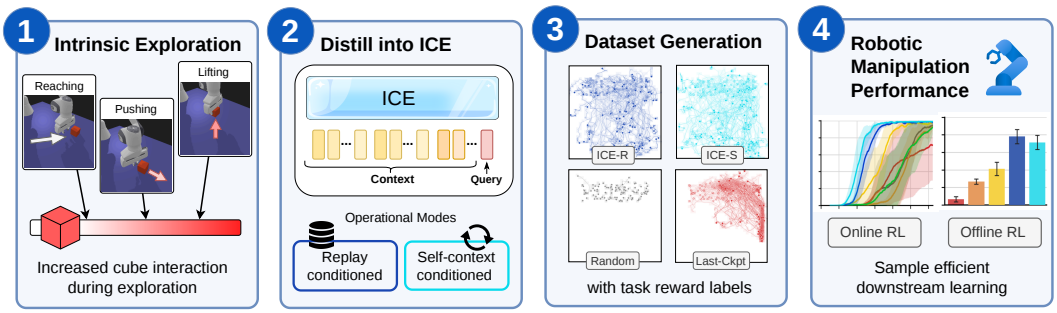


Figure 1: Reward-free data from exploration agents is distilled into the ICE model in order to generate datasets for offline and online reinforcement learning across three robotic manipulation tasks.

of a separate Soft Actor Critic (SAC) [9] agent. Thus, the same task-agnostic exploration prior can support multiple downstream tasks. We evaluate ICE across RND, ICM, and disagreement as source exploration objectives over three manipulation tasks, namely Lift (cube lifting), Push (cube pushing), and Pick-Place (cube pick-and-place), to determine the effect of exploration data for online RL. Furthermore, we train IQL [10] on an offline dataset to isolate task-specific data generation of the exploration methods. We transfer Push policies to the real world, showing improved performance with policies trained on ICE exploration data.

The main contributions of this work are (i) an extensive study of exploration methods for offline and online RL. (ii) An exploration pipeline to effectively learn robotic manipulation tasks, and (iii) a transformer model that conditions on past episodes and changes its context to generate diverse and task-relevant data for efficient reinforcement learning.

2 Related Work

Intrinsic exploration and data reuse. Reward-free exploration separates task-agnostic collection from later optimization for supplied rewards [11]. Following URLB [5] and [12], we distinguish *knowledge-based* methods using prediction error or uncertainty, including ICM [3], RND [2], and Disagreement [4]; *data-based* methods encouraging state-distribution entropy, such as APT [13] and Proto-RL [14]; and *competence-based* methods discovering skills, such as DIAYN [15], APS [16], and the skill-conditioned variant of SMM [17]. APT [13] and DIAYN [15] illustrate the other categories; CIC combines skill-conditioned representation learning with an entropy objective [18]. URLB evaluates pretrained-parameter transfer, whereas ExORL and Explore2Offline relabel collected exploration data for offline RL [6, 7]. ICE shares their separation of collection and task learning but learns a policy that collects *new* trajectories. Our offline evaluation follows this data-utility perspective using IQL [10].

Behavior priors for downstream learning. Behavior priors [19], learned skill priors [20], and OPAL's offline primitives [21] capture recurring interactions to guide task learning or provide temporal abstraction. Play-LMP learns goal-conditioned control from human play [22], while SAC-X schedules auxiliary reward-driven behaviors [23]. ExPLORe [24] and SUPE [25] instead use optimistic reward relabeling to incorporate unlabeled prior experience into online task learning. TMRL [26] broadens pretrained behavior through context noising and learns to modulate that noise during downstream RL. These methods reuse behavior within the task-learning process; ICE uses a frozen collector to populate a separate learner's dataset.

Generative and in-context exploration. Algorithm Distillation [27] uses sequence modeling to predict actions from reward-bearing RL learning histories to reproduce learning in context; DPT [28] predicts optimal-action labels from a query state and interaction context across tasks. ICE builds on this sequence modeling paradigm and adopts history-conditioned prediction to predict exploratory actions without reward inputs in order to warm start downstream task learning. Behavioral Exploration [8] trains a long-context generative policy on demonstrations, conditions on an explicit

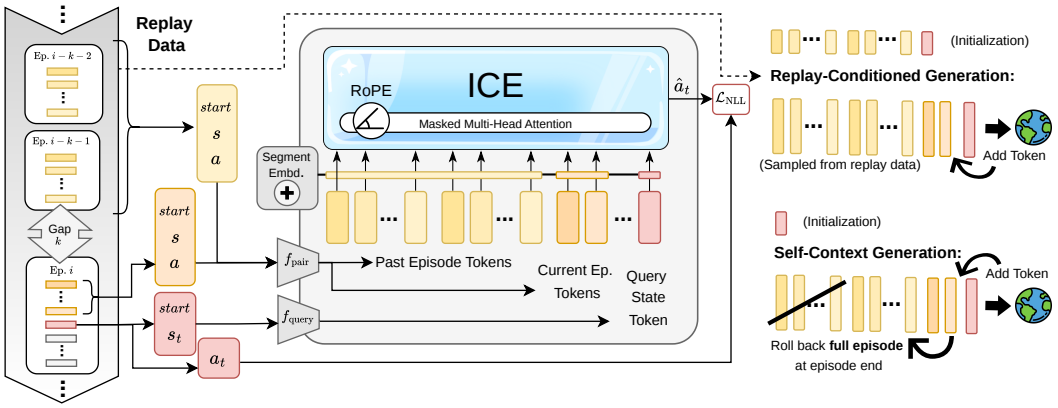


Figure 2: ICE architecture and its replay-conditioned (ICE-R) and self-context (ICE-S) generation modes. Both use the same frozen model. The left shows the conditioning with past episodes, current episodes, and the query state token. The right shows initialization and context changes during rollouts for ICE's operational modes.

measure of additional feature-space coverage relative to history, and feeds back visited states to select exploratory behaviors. ExDM [29] also models heterogeneous exploration replay, but uses diffusion-based state novelty as an intrinsic reward and fine-tunes pretrained policies downstream.

3 Method

Efficient reinforcement learning depends on the agent's ability to explore task-relevant states. Exploration methods gather data from the environment without access to task rewards by maximizing intrinsic rewards. Their behavior changes slowly during exploration. It's unclear how to collect useful data with a limited budget to warm-start online reinforcement learning for robotic manipulation. We define a framework to train an **I**n-**C**ontext **E**xploration (ICE) transformer to generate diverse, task-relevant data for downstream tasks.

Problem Formulation. We define an exploration Markov Decision Process (MDP) as $\mathcal{E} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \rho_0)$, with state space $\mathcal{S}$, action space $\mathcal{A}$, transition dynamics $\mathcal{P}$, and an initial-state distribution ρ_0. An agent explores $\mathcal{E}$ via intrinsic rewards such as RND, ICM, or Disagreement, which depend on learned prediction models and therefore change throughout training. A downstream task defines a goal-conditioned MDP $\mathcal{M}_i = \mathcal{E} \cup (r_i, g_i, \tau_i)$ by adding a goal $g \in \mathcal{G}_i$, reward $r_i(s_t, a_t, s_{t+1}; g_i)$, and success and termination conditions τ_i. Its policy observes (s_t, g) and maximizes $J_i(\pi) = \mathbb{E}_{\pi,g}[\sum_{t=0}^{T_i-1} \gamma^t r_i(s_t, a_t, s_{t+1}; g)]$, where $\gamma \in [0, 1)$ is the discount factor and T_i is the episode length under τ_i. Tasks share system dynamics but have different reward and termination criteria. We aim to train a policy efficiently to maximize $J_i(\pi)$ given a limited exploration data budget. Our framework can be divided into four stages for efficient reinforcement learning in robotic manipulation. We explain the details below, and Figure 1 provides an overview.

Framework. **(1)** Intrinsic Exploration: An exploration agent acts in $\mathcal{E}$ and collects data by optimizing it's intrinsic rewards. Storing its ordered transitions $\mathcal{D}_{\text{explore}} = \{(s_t, a_t, s_{t+1})\}$ and episode boundaries. **(2)** Distill into ICE: We train an in-context exploration transformer via behavioral cloning, conditioned on past episodes, using the previously obtained $\mathcal{D}_{\text{explore}}$ data. The distillation does not depend on reward labels. ICE-model details are explained in the following paragraph. **(3)** Dataset Generation: Collect new trajectories with frozen ICE or source-policy checkpoints and assign downstream rewards and terminal labels; ExORL instead relabels existing exploration replay. **(4)** Robotic Manipulation: Use a budgeted subset for offline IQL or online SAC replay initialization.

Model Architecture. ICE uses state–action tokens as context and a separate query-state token to generate an action. A state–action token also includes an episode-start flag b_t, equal to one at the first step of an episode and zero otherwise. The pair encoder maps each state-action pair $x_t = [b_t, s_t, a_t]$

to $e_t = f_{\text{pair}}(x_t)$; the query encoder maps s_q and b_t to $e_q = f_{\text{query}}([b_t, s_q])$. Learned segment embeddings distinguish past-context tokens, current-episode tokens, and the query token. Let C_{past} be the ordered token sequence from two past episodes and C_{curr} the prefix preceding the query in the current episode. During training, the past-context window and query episode are separated by a fixed offset of 200 episodes in the exploration replay history. Action targets come from a later training stage than the past context and thus promote diverse behavior obtained during training of the intrinsic exploration agent.

Rotary position embeddings (RoPE) [30] encode within-episode order, with position IDs reset at episode boundaries. The query continues the current episode's positional sequence, or starts at its first position when C_{curr} is empty. Our transformer [31, 32] uses causal attention and is trained from scratch with embedding dimension 128, four layers, and four attention heads. The query hidden state h_q parameterizes the action distribution

$$\pi_\theta^{\text{explore}}(a_q \mid s_q, C_{\text{past}}, C_{\text{curr}}) = \mathcal{N}\big(a_q; \mu(h_q), \operatorname{diag}(\sigma^2(h_q))\big)\,.$$

The prediction head outputs the mean and log standard deviation, and training minimizes the Gaussian negative log-likelihood, $\mathcal{L}_{\text{act}} = -\log \pi_\theta(a_q \mid s_q, C_{\text{past}}, C_{\text{curr}})$, with batch size 128 for 30 epochs. Sequences are right-padded, with padding masked from attention. Training uses dropout of 0.1 and current-context dropout of 0.2, which removes the entire current-episode prefix.

Operational Modes. The ICE model samples an action for the current state, executes it in the environment, appends the state–action token to C_{curr}, and queries the next observed state. The current context is cleared at every episode reset. The past episodes' context can be obtained in two ways, influencing the behavior model. The operational modes are explained below.

Replay conditioned (**ICE-R**): At each reset, two episodes sampled from the held-out validation portion of the exploration replay history form C_{past}. This prompt remains fixed during the rollout and is resampled at the beginning of the next episode. The current-episode prefix supplies the newly generated interaction context. Self-context conditioned (**ICE-S**): The first rollout starts with empty past and current contexts. After each episode, its tokens are added to C_{past}, retaining only the two most recent episodes. Subsequent rollouts therefore condition exclusively on ICE's own interaction history and require no replay data at inference. Figure 2 right visualizes the operational modes.

4 Data Analysis and Experiments

Following the framework introduced in Section 3, we describe intrinsic exploration and dataset construction, inspect the resulting trajectories, and evaluate their utility for offline and online learning on three manipulation tasks. Finally, we evaluate direct transfer of simulation-trained task policies to a real robot.

A simulated Franka Panda interacts with a cube (nominal side length 0.0625 m) in a workspace of $x, y \in [-0.20, 0.20]$ and $z \in [0.001, 0.20]$ meters. Episodes last at most 100 control steps at 15 Hz. Five-dimensional actions control normalized relative Cartesian position, wrist yaw, and gripper opening. The 9-dimensional exploration observation comprises normalized end-effector position, sine and cosine of wrist angle, gripper width, and cube position.

4.1 Intrinsic Exploration, Dataset Generation and Analysis

We train separate SAC exploration agents with the RND [2], ICM [3], and dynamics-model disagreement [4] intrinsic exploration objectives for 1 million environment transitions without downstream task rewards. Their curiosity modules are trained from replay, with intrinsic rewards recomputed from the current module before updating the model and normalized by the running standard deviation of replay-sampled intrinsic errors without mean centering. **RND** uses random-feature prediction error on the state excluding end-effector information to promote object interaction, **ICM** uses forward prediction error in a learned feature space, and **Disagreement** uses the prediction variance of an ensemble of five dynamics models. We set intrinsic rewards to zero at workspace boundaries to

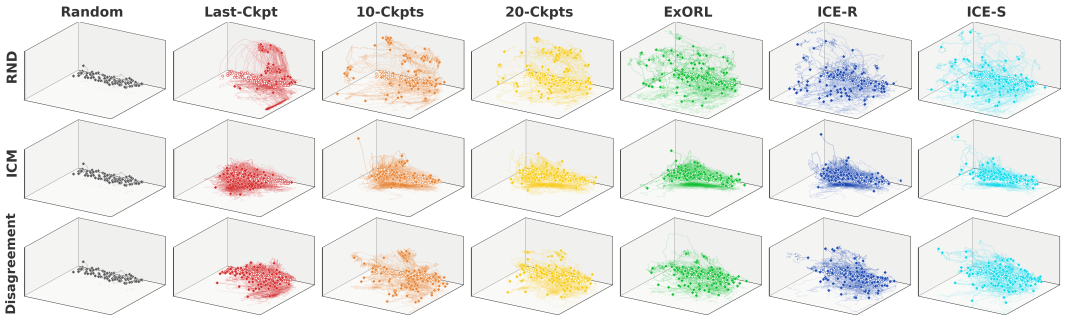


Figure 3: Cube trajectories for each intrinsic objective and data source (100 trajectories per displayed set). Checkpoint policies, ICE, and Random collect new trajectories; ExORL reuses exploration replay. We see diverse behavior, especially compared to random collection. ICE is generating even more diverse trajectories.

discourage the end effector and cube from remaining at the workspace edge to exploit simulation errors. One ICE model is trained on each agent's complete exploration replay history.

For each intrinsic objective, we compare new data collected by source-policy checkpoints and ICE with direct reuse of the source exploration replay. **Last-Ckpt** uses the final exploration-policy checkpoint, while **10-Ckpts** and **20-Ckpts** mix checkpoints distributed across training. **ICE-R** prompts the corresponding ICE model with replay episodes; **ICE-S** uses its own generated history as context. **Random** samples uniformly from the action space to collect data and is independent of the intrinsic objectives. **ExORL** [6] directly reuses transitions from the corresponding source agent's exploration replay, relabeled with downstream rewards and termination conditions. Each dataset contains 150,000 transitions, from which we select task-learning subsets. Checkpoint mixtures contain equal numbers of complete episodes from each checkpoint; ExORL samples trajectories uniformly across the replay history. Unlike the original ExORL approach, which reuses the full exploration replay buffer, we uniformly subsample 150,000 transitions from the replay history to match the dataset size used by the other methods. In offline experiments we train policies with IQL [10]; in online experiments the data initializes SAC replay buffer before interaction. Within each task and learning setting, methods receive the same downstream transition budget.

Figure 3 shows 100 cube trajectories from each data source. Final-checkpoint rollouts often form distinct spatial clusters and repeated motion patterns, particularly for RND. Checkpoint mixtures, exploration replay, and ICE contain a broader range of these behaviors, while ICE-R shows substantial spatial spread for RND and Disagreement, while ICM trajectories remain more localized. These samples suggest that ICE inherits characteristics of its underlying source exploration data. We evaluate the utility of each dataset separately through downstream learning.

4.2 Downstream Robotic Manipulation Tasks

We consider three downstream robotic manipulation tasks derived from the same exploration environment. Each task adds the goal state to the observation, shares the initial state distribution, and terminates upon task success. Figure 4 shows the task setup on the left.

PUSH requires moving the cube within 2 cm of a target ≈15 cm away from its starting position.
LIFT succeeds when the cube reaches a sampled target height between 10 cm and 20 cm.
PICK-PLACE requires positioning the cube within 2 cm of a goal located on a platform with various heights. In addition to the initial robot and cube poses, we randomize the cube's size and friction.

Task rewards combine a success bonus of 10, a per-step penalty of -0.12, an cube out-of-bounds terminal penalty of -10, and a small end-effector boundary penalty with potential-based shaped [33] reward terms: $r'_i(s_t, a_t, s_{t+1}; g) = r_{\text{sparse}}(s_t, a_t, s_{t+1}; g) + \alpha\left[\gamma\Phi_i(s_{t+1}; g) - \Phi_i(s_t; g)\right]$,

where $r_{i,\text{sparse}}$ contains the bonuses and penalties above, $\alpha = 1$, and $\gamma = 0.99$. The potentials combine end-effector to cube proximity with task progress: horizontal cube-to-goal distance for

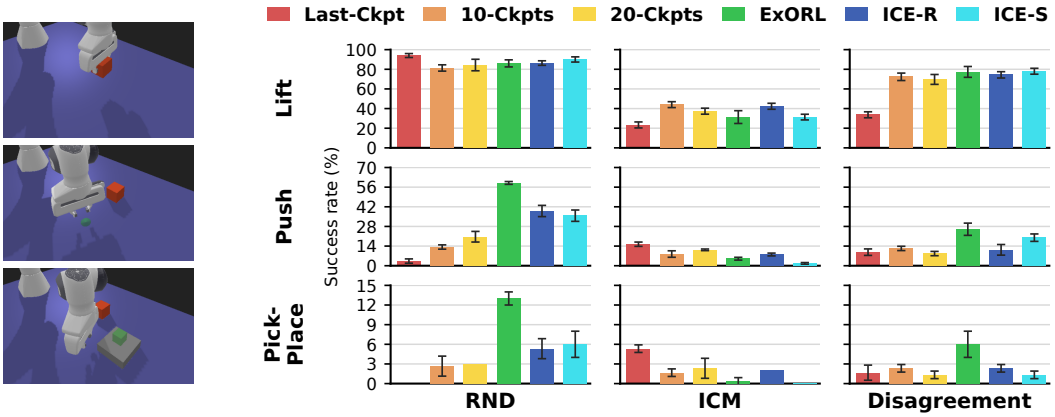


Figure 4: Simulation setup for Lift, Push, and Pick-Place, with goals in green (left). Offline RL (right): Downstream IQL maximum success rates reached using 70,000 transitions for Lift and 140,000 for Push and Pick-Place, grouped by task and intrinsic exploration objective (right). Error bars show the standard deviation across three seeds of each seed's best evaluation success rate.

pushing, vertical progress for lifting, and 3-D goal distance plus lifting progress for Pick-Place. Distances are normalized by their episode-initial values.

Offline RL. We train IQL [10] on data from every exploration method across every intrinsic objective. Concurrent advantage-weighted regression (AWR) policy extraction is used without online interaction. Within each task, all methods and variants receive the same transition budget: 70,000 for Lift and 140,000 each for Push and Pick-Place. We report the best observed evaluation success during 250,000 update steps over three seeds, evaluated every 5,000 updates with 100 trials.

Figure 4 shows that direct replay reuse is a strong offline baseline. ExORL achieves the highest displayed mean success for both RND- and Disagreement-based Push and Pick-Place. In particular, RND with ExORL reaches approximately 60% success on Push and 13% on Pick-Place, compared with approximately 40% and 5%, respectively, for ICE-R. Thus, ICE does not consistently improve on reusing the source replay data, although both ICE variants outperform checkpoint-collected data for RND-based Push. For ICM-based Push, Last-Ckpt remains the strongest baseline. Results on Lift are also source-dependent: RND datasets perform well, ICM checkpoint mixtures and ICE-R outperform the final checkpoint, and Disagreement-based replay, checkpoint mixtures, and ICE substantially exceed its final checkpoint. Pick-Place remains difficult across all datasets, including ExORL. Since offline and online results use different learners and budgets, they do not isolate the effect of permitting online interaction.

Online RL. We warm-start an online RL agent (SAC [9]) with exploration data from each variant showcasing its usefulness for efficient robotic manipulation. The initial replay buffer is filled with 50,000 samples excluding post-success transitions. Every agent is warm-started with 1,000 gradient update steps before interaction with the environment and has the same budget and hyperparameters.

We evaluate each method and its variants on Lift, Push, and Pick-Place over five seeds, grouping the exploration-based methods by intrinsic exploration objective (RND, ICM, Disagreement). Figure 5 reports downstream task success rates over environment transitions, including the initialization data; shaded regions indicate $\pm$ one standard deviation across independent runs.

For a compact online learning performance comparison, we follow the use of learning-curve area in RL transfer evaluation [34] and report the normalized area under the success-rate curve (nAUC).

$$A_{m,i,j} = \frac{1}{H_i - a_i} \int_{a_i}^{H_i} p_{m,i,j}(x)\,\mathrm{d}x, \tag{1}$$

where $p_{m,i,j}(x) \in [0,1]$ denotes training success for method m, task i, and intrinsic objective j after x downstream environment steps. All runs for a task share the same interval $[a_i, H_i]$. Higher

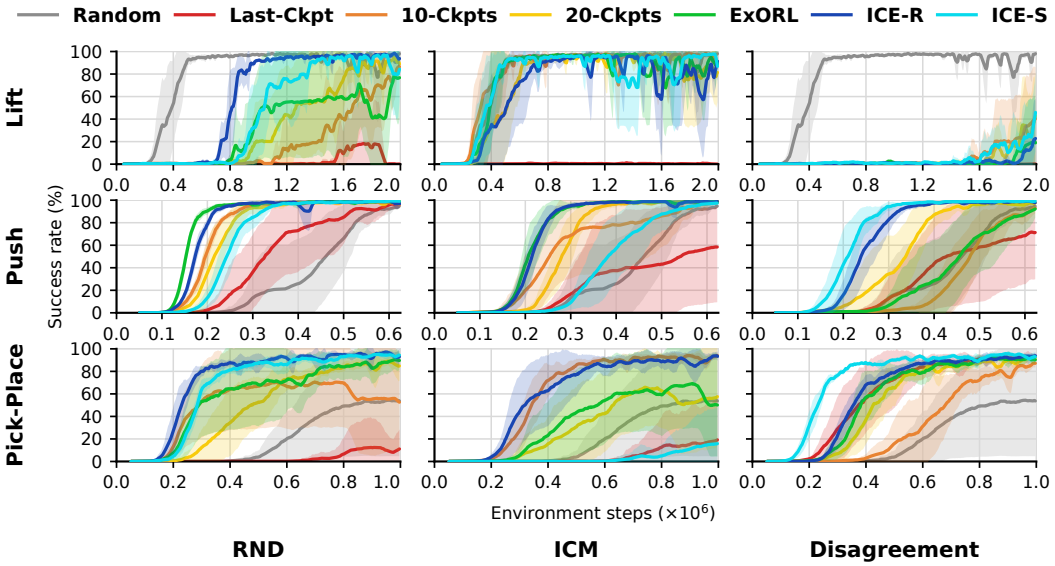


Figure 5: Online RL results on Lift, Push, and Pick-Place tasks across RND-, ICM-, and Disagreement-based exploration methods; all conditions use 50,000-transition replay initialization. Exploration data leads to efficient task learning, whereas ICE further accelerates performance. We show the mean success rate and ± one standard deviation over five seeds.

Table 1: Normalized area under the online training success-rate curve (nAUC; higher is better). Task rows average over intrinsic objectives, objective rows over tasks, and Overall over all nine conditions, after averaging across seeds. Random is independent of the intrinsic objective. Bold and italic indicate the highest and second-highest displayed values per row.

	Random	Last-Ckpt	10-Ckpts	20-Ckpts	ExORL	ICE-R	ICE-S
Lift	**0.80**	0.01	0.35	0.36	0.36	*0.43*	0.42
Push	0.31	0.34	0.51	*0.60*	*0.60*	**0.70**	0.58
Pick-Place	0.20	0.22	0.49	0.44	*0.51*	**0.66**	0.48
RND	0.44	0.18	0.48	0.50	0.57	**0.70**	*0.59*
ICM	0.44	0.10	**0.67**	0.54	*0.61*	**0.67**	0.40
Disagreement	*0.44*	0.29	0.21	0.36	0.29	0.42	**0.50**
Overall	0.44	0.19	0.45	0.47	0.49	**0.59**	*0.50*

nAUC indicates greater average success over this interval, rewarding earlier improvement and sustained performance. We compute nAUC per seed before averaging across seeds; task and objective summaries average over the other dimension, and the overall score weights all nine task–objective conditions equally.

Downstream performance depends on the intrinsic objective and task, with no method dominating all settings. Random data performs best on Lift, achieving an nAUC of 0.80 compared with 0.43 for ICE-R (Table 1). For Push and Pick-Place, checkpoint mixtures generally accelerate learning relative to Last-Ckpt, while ICE provides further gains in several settings. For RND pushing, ICE reaches near-optimal success approximately twice as fast as Random, despite similar final performance (Figure 5). For RND-based Pick-Place, both ICE variants exceed approximately 80% success by 400,000 environment steps, while Random remains near zero. ExORL is competitive in several settings, but ICE-R learns faster for Disagreement-based Push and reaches higher success for ICM-based Pick-Place. These gains are reflected in the task-averaged nAUC: ICE-R leads on Push (0.70) and Pick-Place (0.66), exceeding the strongest non-ICE baselines by 0.10 and 0.15, respectively. Overall, ICE-R ranks highest at 0.59, followed by ICE-S (0.50) and ExORL (0.49). Across intrinsic objectives, ICE-R leads for RND and ties 10-Ckpts for ICM, whereas ICE-S leads for Disagree-

ment. Together, the learning curves and aggregate scores show that ICE-generated initialization can improve downstream learning efficiency, with the largest benefits on Push and Pick-Place.

4.3 Real World Evaluation

We directly transfer simulation-trained Push policies to a real Franka FR3, comparing with all policies trained on the five RND- and two ICE-variants. We randomize end-effector position by ± 5 cm in x and ± 2.5 cm in y; we also randomize cube position within 4 cm and its yaw orientation. We track the cube with two cameras and FoundationPose [35] to determine its state and the success criteria. We evaluate checkpoints every 100,000 training steps from 200,000 to 600,000 using ten trials per seed across three seeds, totaling 30 trials. Figure 6 shows the task setup and randomization on the left, and we report the mean success rate and 95% confidence interval on the right.

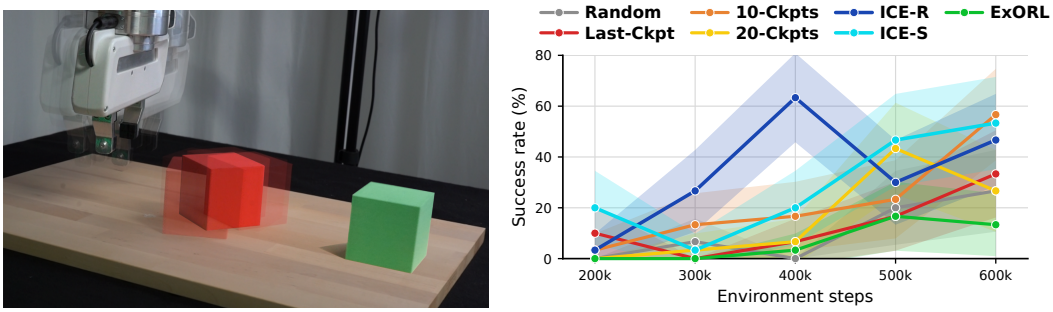


Figure 6: Real robot Push setup, with the goal in green and reset regions overlaid (left). Success rates of simulation-trained task policies using Random, RND-based variants, and ICE (right). We evaluate checkpoints from 200,000 to 600,000 environment steps with 30 trials across three seeds; shaded intervals indicate 95% confidence. Policies trained with ICE exploration data show better performance, while policies trained without exploration data have the lowest success rate.

Because of the sim-to-real gap, policy performance decreases compared to the simulation results. ICE-R has the highest success rate after 300,000 and 400,000 training steps but falls off afterward; ICE-S achieves the best final performance of 55% with RND 10-Ckpts. ICE variants show better sim-to-real transfer only by providing exploration data. ExORL remains lower in this transfer evaluation. Early policies often reach the cube without completing a push, while failure cases for more trained policies mainly involve poor lateral contact, misaligned pushes, or failure to recover near the workspace boundary. We plan to evaluate policies in more tasks to investigate the influence of exploration data on downstream task performance and sim-to-real transfer.

5 Conclusion

In this work, we introduced In-Context Exploration (ICE), a framework to distill reward-free exploration data into a context-conditioned transformer for efficient robotic manipulation. We evaluated ICE with three exploration methods across three manipulation tasks in offline and online RL settings. We show that suitable exploration data leads to efficient training and that ICE can boost performance in simulation and sim-to-real transferred policies. In future work, we want to further evaluate policies on more tasks and use ICE to collect data directly on the real system for sample-efficient real-world reinforcement learning.

Acknowledgments

This research is funded by the German Research Foundation (DFG) Emmy Noether Programme (CH 2676/1-1), the EU's Horizon Europe project ARISE (Grant no.: 101135959), the German Federal Ministry of Education and Research (BMBF) project "RiG" (Grant no.: 16ME1001) and the European Research Council (ERC) project "SIREN" (Grant No.: 101163933). The hardware is funded by the German Research Foundation (DFG). The authors gratefully acknowledge the computing time provided to them on the high-performance computer Lichtenberg II at TU Darmstadt, funded by the German Federal Ministry of Research, Technology and Space (BMFTR) and the State of Hesse.

References

[1] G. Dulac-Arnold, D. Mankowitz, and T. Hester. Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901*, 2019.

[2] Y. Burda, H. Edwards, A. Storkey, and O. Klimov. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*, 2018.

[3] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017. URL https://proceedings.mlr.press/v70/pathak17a.html.

[4] D. Pathak, D. Gandhi, and A. Gupta. Self-supervised exploration via disagreement. In *International conference on machine learning*, pages 5062–5071. PMLR, 2019. URL https://proceedings.mlr.press/v97/pathak19a.html.

[5] M. Laskin, D. Yarats, H. Liu, K. Lee, A. Zhan, K. Lu, C. Cang, L. Pinto, and P. Abbeel. Urlb: Unsupervised reinforcement learning benchmark. *arXiv preprint arXiv:2110.15191*, 2021. URL https://arxiv.org/abs/2110.15191.

[6] D. Yarats, D. Brandfonbrener, H. Liu, M. Laskin, P. Abbeel, A. Lazaric, and L. Pinto. Don't change the algorithm, change the data: Exploratory data for offline reinforcement learning, 2022. URL https://arxiv.org/abs/2201.13425.

[7] N. Lambert, M. Wulfmeier, W. Whitney, A. Byravan, M. Bloesch, V. Dasagi, T. Hertweck, and M. Riedmiller. The challenges of exploration for offline reinforcement learning. *arXiv preprint arXiv:2201.11861*, 2022. URL https://arxiv.org/abs/2201.11861.

[8] A. Wagenmaker, Z. Zhou, and S. Levine. Behavioral exploration: Learning to explore via in-context adaptation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61885–61912. PMLR, 2025. URL https://proceedings.mlr.press/v267/wagenmaker25a.html.

[9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870. PMLR, 2018.

[10] I. Kostrikov, A. Nair, and S. Levine. Offline reinforcement learning with implicit q-learning, 2021. URL https://arxiv.org/abs/2110.06169.

[11] C. Jin, A. Krishnamurthy, M. Simchowitz, and T. Yu. Reward-free exploration for reinforcement learning, 2020. URL https://arxiv.org/abs/2002.02794.

[12] A. Srinivas and P. Abbeel. Unsupervised learning for reinforcement learning. In *Tutorial in the 9th International Conference on Learning Representations (ICLR'21)*, 2021. URL https://icml.cc/media/icml-2021/Slides/10843_QHaHBNU.pdf.

[13] H. Liu and P. Abbeel. Behavior from the void: Unsupervised active pre-training. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/99bf3d153d4bf67d640051a1af322505-Abstract.html.

[14] D. Yarats, R. Fergus, A. Lazaric, and L. Pinto. Reinforcement learning with prototypical representations. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 11920–11931. PMLR, 2021. URL https://proceedings.mlr.press/v139/yarats21a.html.

[15] B. Eysenbach, A. Gupta, J. Ibarz, and S. Levine. Diversity is all you need: Learning skills without a reward function. *arXiv preprint arXiv:1802.06070*, 2018.

[16] H. Liu and P. Abbeel. Aps: Active pretraining with successor features. In *International Conference on Machine Learning*, pages 6736–6747. PMLR, 2021.

[17] L. Lee, B. Eysenbach, E. Parisotto, E. Xing, S. Levine, and R. Salakhutdinov. Efficient exploration via state marginal matching. *arXiv preprint arXiv:1906.05274*, 2019. URL https://arxiv.org/abs/1906.05274.

[18] M. Laskin, H. Liu, X. B. Peng, D. Yarats, A. Rajeswaran, and P. Abbeel. Unsupervised reinforcement learning with contrastive intrinsic control, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/debf482a7dbdc401f9052dbe15702837-Abstract-Conference.html.

[19] D. Tirumala, A. Galashov, H. Noh, L. Hasenclever, R. Pascanu, J. Schwarz, G. Desjardins, W. M. Czarnecki, A. Ahuja, Y. W. Teh, and N. Heess. Behavior priors for efficient reinforcement learning. *Journal of Machine Learning Research*, 23(221):1–68, 2022. URL https://www.jmlr.org/papers/v23/20-1038.html.

[20] K. Pertsch, Y. Lee, and J. Lim. Accelerating reinforcement learning with learned skill priors, 2021. URL https://proceedings.mlr.press/v155/pertsch21a.html.

[21] A. Ajay, A. Kumar, P. Agrawal, S. Levine, and O. Nachum. Opal: Offline primitive discovery for accelerating offline reinforcement learning, 2020. URL https://arxiv.org/abs/2010.13611.

[22] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet. Learning latent plans from play. In *Conference on robot learning*, pages 1113–1132. Pmlr, 2020.

[23] M. Riedmiller, R. Hafner, T. Lampe, M. Neunert, J. Degrave, T. Wiele, V. Mnih, N. Heess, and J. T. Springenberg. Learning by playing solving sparse reward tasks from scratch. In *International conference on machine learning*, pages 4344–4353. PMLR, 2018.

[24] Q. Li, J. Zhang, D. Ghosh, A. Zhang, and S. Levine. Accelerating exploration with unlabeled prior data. *Advances in Neural Information Processing Systems*, 36:67434–67458, 2023. URL https://arxiv.org/abs/2311.05067.

[25] M. Wilcoxson, Q. Li, K. Frans, and S. Levine. Leveraging skills from unlabeled prior data for efficient online exploration. *arXiv preprint arXiv:2410.18076*, 2024. URL https://arxiv.org/abs/2410.18076.

[26] M. M. Hong, J. Zhang, A. Nagabandi, and A. Gupta. Tmrl: Diffusion timestep-modulated pretraining enables exploration for efficient policy finetuning, 2026. URL https://arxiv.org/abs/2605.12236.

[27] M. Laskin, L. Wang, J. Oh, E. Parisotto, S. Spencer, R. Steigerwald, D. Strouse, S. Hansen, A. Filos, E. Brooks, et al. In-context reinforcement learning with algorithm distillation. *arXiv preprint arXiv:2210.14215*, 2022. URL https://arxiv.org/abs/2210.14215.

[28] J. Lee, A. Xie, A. Pacchiano, Y. Chandak, C. Finn, O. Nachum, and E. Brunskill. Supervised pretraining can learn in-context reinforcement learning. *Advances in Neural Information Processing Systems*, 36:43057–43083, 2023. URL https://arxiv.org/abs/2306.14892.

[29] C. Ying, H. Chen, X. Zhou, Z. Hao, H. Su, and J. Zhu. Exploratory diffusion model for unsupervised reinforcement learning. *arXiv preprint arXiv:2502.07279*, 2025. URL https://arxiv.org/abs/2502.07279v2.

[30] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[31] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[32] M. Janner, Q. Li, and S. Levine. Offline reinforcement learning as one big sequence modeling problem. *Advances in neural information processing systems*, 34:1273–1286, 2021.

[33] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Icml*, volume 99, pages 278–287. Citeseer, 1999.

[34] M. E. Taylor and P. Stone. Transfer learning for reinforcement learning domains: A survey. *Journal of Machine Learning Research*, 10(56):1633–1685, 2009. URL https://www.jmlr.org/papers/v10/taylor09a.html.

[35] B. Wen, W. Yang, J. Kautz, and S. Birchfield. Foundationpose: Unified 6d pose estimation and tracking of novel objects. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17868–17879. IEEE, 2024.